Highlights

Steering Through the Trees: Physical Lie Vector Field Evasion of Tree Ensembles in Network Intrusion Detection

Quang-Vinh Dang

- Four packet-level attacker actions are modeled as Lie-cone generators on flows.
- Trajectories in the cone provably preserve rate, length, header and MTU invariants.
- 100% real attack success at 12.9 queries; baselines reach 48–82% at 1,240–1,850.
- Success stays at 100% on three calibrated corpora with 6.8–17.8 queries per flow.
- Defenses cut FFE success from 95% to 56% (non-adaptive attacker, M=5).

Steering Through the Trees: Physical Lie Vector Field Evasion of Tree Ensembles in Network Intrusion Detection

Quang-Vinh Dang[a,*]

[a]*School of Innovative and Computing Sciences, British University Vietnam, Hung Yen, Vietnam*

Abstract

Gradient-boosted tree ensembles dominate flow-based network intrusion detection, yet their robustness to evasion that respects flow semantics is unclear. Feature-space attacks ignore the coupling between flow features and yield unrealizable flows, whereas packet-level heuristics treat the detector as a black box and need thousands of queries. We propose Flow-Field Evasion (FFE), which models four attacker-controlled actions (temporal dilation, payload padding, chaff injection and TCP window morphing) as generators of a Lie cone on the flow manifold. We prove that trajectories in the continuous relaxation of this cone preserve the rate, length, header and MTU invariants, and a projection enforces them on the discrete flow schema. FFE follows the generator with the largest cost-weighted score decrease, then snaps to tree-split boundaries. Against CUDA XGBoost on a corpus calibrated to CSE-CIC-IDS2018, FFE reaches 100% real attack success with 12.9 queries per flow, versus 48.0% and 82.0% for CAA/MOEVA and PSO at 1,240 and

*Corresponding author

Email address: vinh.dq4@buv.edu.vn (Quang-Vinh Dang)

1,850 queries. On corpora calibrated to UNSW-NB15 and CIDDS-001 and against LightGBM, it stays at 100% with 6.8–17.8 queries. Feature masking with single-round adversarial training lowers success from 95.0% to 56.0% against a non-adaptive attacker (five-step budget), at a 2.42-point accuracy cost.

Keywords: Adversarial machine learning, Network intrusion detection, Gradient boosted decision trees, Evasion attacks, Problem-space constraints, Lie algebra, Adversarial training

1. Introduction

Modern network intrusion detection systems (NIDS) increasingly rely on machine learning over flow-based telemetry. At multi-gigabit rates, inspecting every packet payload is prohibitive, so sensors such as CICFlowMeter [1] aggregate bidirectional packet exchanges into tabular flow records of timing, length, header and flag statistics. Gradient-boosted decision trees (GBDTs), notably XGBoost [2], LightGBM [3] and CatBoost [4], are the dominant classifiers for these records.

Near-perfect accuracy on benchmark corpora does not guarantee robustness in operation. Closed-world evaluations reward models that exploit testbed artifacts [5] and shortcut features [6]. Attackers, however, control the malleable parts of their traffic, such as timing, padding and window sizes, so a detector that relies on these features can be evaded without losing the exploit [7].

Evading a tree ensemble is hard for two reasons. First, the ensemble is piecewise constant, so $\nabla_x F(x) = 0$ for almost all x and gradient-based

attacks receive no signal. Second, flow features obey conservation laws. Flow Bytes/s satisfies $x_8 = 10^6(x_4 + x_5)/x_6$, where x_4 and x_5 are the forward and backward byte totals and x_6 is the duration in microseconds. A perturbation that changes the byte totals without updating the rate produces a phantom flow that protocol sanity checks reject [8, 7].

Existing work follows one of two paradigms, and neither is adequate. Feature-space methods such as ZOO [9] and HopSkipJump [10] ignore the coupling: their Physical Feasibility Rate (PFR) falls to 17.8–25.0% and their real attack success reaches at most 8.0%, and HopSkipJump needs 680 queries per flow. Packet-space approaches, including Traffic Manipulator [11] and constraint-aware search [12, 13], produce routable traffic but query the classifier as a black box. Their real attack success is 48.0% for CAA/MOEVA and 82.0% for PSO, at 1,240 and 1,850 queries respectively.

This article proposes Flow-Field Evasion (FFE), which combines both properties. FFE derives physical perturbations as differential generators, so the continuous flow satisfies the invariants exactly and a projection enforces them on the discrete schema, and it steers flows using the split geometry of the ensemble. To our knowledge, no prior attack on tree ensembles combines these two properties.

The main contributions are as follows.

- We model four packet-level attacker actions (temporal dilation, payload padding, chaff injection and TCP window morphing) as closed-form infinitesimal generators of a Lie cone on the flow manifold.
- We prove that trajectories in this cone preserve the rate, mean-length, header and MTU invariants of flow records in the continuous relaxation

of the flow manifold.

- We present FFE, which moves each flow along the generator with the largest cost-weighted score decrease and finishes with tree-split boundary snapping (TSBS).

- We evaluate FFE on calibrated corpora derived from CSE-CIC-IDS2018, UNSW-NB15 and CIDDS-001. It reaches 100% real attack success with 12.9 queries per flow against XGBoost on CSE-CIC-IDS2018, and 100% with 6.8–17.8 queries across all six dataset and model cohorts. Malleability-penalized splitting combined with Lie-guided adversarial training reduces FFE success from 95.0% to 56.0% under a five-step attack budget.

2. Related Work

Gradient-boosted decision trees (GBDTs), including XGBoost [2], LightGBM [3] and CatBoost [4], are the standard choice for flow-based network intrusion detection on benchmarks such as CSE-CIC-IDS2018 [1], UNSW-NB15 [14] and CIDDS-001 [15]. Comprehensive empirical studies support this choice [16, 17, 7]. Their split rules depend only on the rank order of feature values and are cheap to evaluate, which makes them attractive for deployment. Learned detectors have long been questioned for operation outside the closed world of their training data [5], but robustness to realistic evasion has received less attention.

Early adversarial attacks on NIDS were transplanted from image classification. Szegedy et al. [18], Goodfellow et al. [19] and Papernot et al. [20]

showed that gradient-based perturbations fool deep networks, and optimizers such as the Carlini-Wagner L_2 attack [21] were then applied to tabular flow features. These studies reported high success rates, but the perturbed vectors were not network flows. Continuous optimization in Euclidean space assigns fractional values to integer packet counts, modifies immutable fields such as destination port, produces negative durations, and breaks the conservation between byte totals and packet counts. A feature vector that no valid packet sequence can produce says nothing about the deployed detector.

The problem-space view corrects this. Pierazzi et al. [8] showed that an adversarial perturbation is meaningful only when it is realized as a transformation of the physical object, such as a binary or a packet sequence, and that the transformation must preserve functionality and survive the feature extractor. Verkerken et al. [22] apply this view to defining host-space adversarial perturbations against NIDS. Surveys and audits of this literature [7, 23, 24, 25] argue that many feature-space attacks assume threat models that cannot be realized with valid packets under RFC protocol specifications, so their success rates overstate the risk once evaluated against true packet parsers and middleboxes. Constrained tabular attacks address feasibility only in part. CAA [12] combines a gradient-based attack (CAPGD) with MOEVA [13], a multi-objective genetic search whose constraint penalties are heuristic and whose population search receives no directional guidance. TabularBench [26] standardizes such evaluations across deep and tree models. Packet-level tools avoid these problems by operating on real traffic. Traffic Manipulator [11] uses particle swarm optimization to mutate PCAP files by perturbing inter-packet intervals, adding payload bytes and injecting

chaff packets. IDSGAN [27] generates adversarial flow records with generative adversarial networks, but its feasibility is only approximated in feature space.

Exact methods exploit the piecewise-constant structure of trees. Kantchelian et al. [28] cast minimal evasion against tree ensembles as a mixed-integer linear program. The formulation is exact but NP-hard in general and accepts only linear constraints, so it cannot express the nonlinear rate relations between bytes and duration that flow records obey. SAT and MaxSAT encodings of tree ensembles [29, 30] provide formal explanations and robustness checks, and LT-Attack [31] attacks tree ensembles through continuous leaf approximations. The MILP and LT-Attack operate in unconstrained feature space and do not enforce coupled network invariants. Robust training of tree ensembles [32, 33, 34] and adversarial retraining of neural NIDS [35] improve robustness within the same feature-space threat model. Black-box query methods such as ZOO [9] and HopSkipJump [10] estimate the decision boundary by sampling the classifier's output, and population or swarm searches mutate parameters without knowing which splits are active.

Table 1 summarizes these paradigms and the gap they leave. Methods that exploit the structure of tree ensembles or work in unconstrained feature space do not produce flows that survive packet parsing, while methods that preserve physical feasibility treat the detector as a black box and need hundreds to thousands of queries per flow. Flow-Field Evasion (FFE) addresses this gap by steering flows along infinitesimal packet-level generators that keep every step on the flow manifold, and by using the split geometry of the trees to choose the direction.

3. Problem Formulation and Threat Model

3.1. Valid Packet Streams and Flow Telemetry

A packet stream $P = ((p_1, t_1), \ldots, (p_m, t_m))$ is an ordered sequence of packets with $0 \leq t_1 \leq \cdots \leq t_m$, where t_i is the transmission time at the network interface card. Only streams in the admissible set $\mathcal{P}_{\text{valid}}$ are considered. Every packet must carry valid IPv4, TCP and UDP checksums [36, 37, 38] and obey the interface MTU:

$$\forall i \in \{1, \ldots, m\}, \quad \text{Len}_{\text{IP}}(p_i) \leq \text{MTU}_{\text{IP}} = 1500 \text{ bytes} \tag{1}$$

A flow is the bidirectional set of packets sharing a 5-tuple, split into forward (client-to-server) and backward packets. A flow exporter such as CICFlowMeter [1] computes a deterministic, many-to-one mapping

$$\Phi : \mathcal{P}_{\text{valid}} \to \mathbb{R}^d \tag{2}$$

and $x = \Phi(P)$ summarizes the transmission.

3.2. Twelve-Dimensional Feature Schema

We use a standardized 12-dimensional schema of the core metrics that state-of-the-art models analyze on CSE-CIC-IDS2018 [1]. The feature vector $x \in \mathbb{R}^{12}$ comprises: x_0, destination port; x_1, protocol (6 for TCP, 17 for UDP); x_2, number of forward packets; x_3, number of backward packets; $x_4 = \sum_{p_i \in P_{\text{fwd}}} \text{Len}(p_i)$, forward wire bytes including IP and transport headers; x_5, backward bytes; $x_6 = (t_m - t_1) \times 10^6$, flow duration in microseconds; x_7, initial forward TCP window ($0 \leq x_7 \leq 65535$, and $x_7 \equiv 0$ for UDP); x_8, flow bytes per second; x_9, flow packets per second; x_{10}, mean forward packet

length; and x_{11}, forward transport header bytes, which must be at least the per-packet minimum header size times x_2. The rates and mean are fixed by the relation

$$x_8 = g_8(x_4, x_5, x_6) = \frac{10^6\,(x_4 + x_5)}{x_6}, \qquad x_9 = g_9(x_2, x_3, x_6) = \frac{10^6\,(x_2 + x_3)}{x_6},$$
$$x_{10} = g_{10}(x_4, x_2) = \frac{x_4}{\max(1, x_2)} \tag{3}$$

3.3. Feature Invariants

The indices $\mathcal{D} = \{0, \ldots, 11\}$ fall into five constraint classes derived from network semantics, and a feature may belong to more than one class.

- $\mathcal{I}_{\text{imm}} = \{0, 1, 3, 5\}$: port, protocol and backward packets and bytes are fixed by the service and the server, so $\Delta x_i = 0$.
- $\mathcal{I}^{+}_{\text{mono}} = \{2, 4, 6, 11\}$: an attacker may add packets, pad payloads or delay transmissions, but cannot remove bytes the payload needs, so $\Delta x_j \geq 0$.
- $\mathcal{I}_{\text{bound}} = \{7\}$: the window stays within $0 \leq x_7 + \Delta x_7 \leq 65535 \cdot \mathbb{I}_{x_1=6}$.
- $\mathcal{I}_{\text{dep}} = \{8, 9, 10\}$: these features are the deterministic functions g_8, g_9, g_{10} of the base metrics given above.
- $\mathcal{I}_{\text{ineq}} = \{4, 11\}$: $x_{11} \geq x_2(20\,\mathbb{I}_{x_1=6} + 8\,\mathbb{I}_{x_1=17})$ and $x_{11} + 20x_2 \leq x_4 \leq 1500x_2$.

Feature x_4 is both monotone and MTU bounded. The set $\mathcal{I}^{+}_{\text{mono}}$ gives the mutable monotone features that Algorithm 2 modifies.

3.4. Flow Manifold and Ambient Relaxation

The physical flow space is the discrete image of valid packet streams under the telemetry exporter:

$$\mathcal{M}_{\text{flow}} = \left\{ x \in \mathbb{R}^{12} \;\middle|\; \exists P \in \mathcal{P}_{\text{valid}} \text{ s.t. } \Phi(P) = x \right\} \subset \mathbb{Z}^9_{\geq 0} \times \mathbb{R}^3_{\geq 0} \tag{4}$$

where discrete coordinates $(x_0, x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_{11})$ satisfy integrality (with packet counts $x_2 \in \mathbb{N}_{\geq 1}, x_3 \in \mathbb{N}_{\geq 0}$, duration in integer microseconds $1 \leq x_6 \leq 1.1 \times 10^8$ μs bounded below the 120-second exporter timeout), rates $(x_8, x_9, x_{10}) \in \mathbb{R}^3_{\geq 0}$, and packet lengths comply with interface MTU restrictions ($x_4 \leq 1500 x_2$). We focus on IPv4 ($L_{\text{IPv4}} = 20$ bytes) and standard Ethernet MTU.

To enable continuous differential geometric optimization, we define the *continuous ambient flow manifold* $\widetilde{\mathcal{M}}_{\text{flow}} \subset \mathbb{R}^{12}$ by relaxing packet counts and duration to positive reals ($x_2 \geq 1, x_6 > 0$), with transport header limits $H_{\min}(x_1) = 20 \cdot \mathbb{I}_{x_1=6} + 8 \cdot \mathbb{I}_{x_1=17}$ and $H_{\max}(x_1) = 60 \cdot \mathbb{I}_{x_1=6} + 8 \cdot \mathbb{I}_{x_1=17}$:

$$\widetilde{\mathcal{M}}_{\text{flow}} = \left\{ x \in \mathbb{R}^{12} \;\middle|\; \begin{array}{l} x_0 \in [1, 65535], \quad x_1 \in \{6, 17\}, \quad x_2 \geq 1, \quad x_3, x_5 \geq 0, \\ H_{\min}(x_1) x_2 \leq x_{11} \leq H_{\max}(x_1) x_2, \\ x_{11} + 20 x_2 \leq x_4 \leq 1500 x_2, \quad 0 < x_6 \leq 1.1 \times 10^8, \\ 0 \leq x_7 \leq 65535 \cdot \mathbb{I}_{x_1=6}, \quad x_{10} = x_4 / x_2, \\ x_8 = 10^6 (x_4 + x_5)/x_6, \quad x_9 = 10^6 (x_2 + x_3)/x_6 \end{array} \right\} \tag{5}$$

Continuous vector field flows are formulated on $\widetilde{\mathcal{M}}_{\text{flow}}$, while discrete execution snaps continuous trajectories back to $\mathcal{M}_{\text{flow}}$ via the closed-form projection operator $\mathcal{P}_{\mathcal{M}}$ (Section 4.4).

3.5. Threat Model

The adversary starts from a malicious flow $x_{\text{orig}} = \Phi(P) \in \mathcal{M}_{\text{flow}}$ classified as an attack, $P(\text{Malicious} \mid x_{\text{orig}}) \geq 0.5 \iff F(x_{\text{orig}}) \geq 0$, and seeks a modified flow $x^* = \Phi(P^*) \in \mathcal{M}_{\text{flow}}$ with

$$P(\text{Malicious} \mid x^*) < 0.5 \iff F(x^*) < 0 \tag{6}$$

Tier 1 (white-box) grants access to the ensemble architecture, leaf weights and split thresholds Θ_j, which enables exact Tree-Split Boundary Snapping (TSBS) and forms the primary benchmark (Section 6). Tier 2 (black-box) allows only query access or zero-knowledge transfer of flows crafted on a surrogate ensemble, which Table 5 evaluates.

The transformed trace P^* must preserve the malicious semantics, $\mathcal{S}(P^*) = \mathcal{S}(P)$, so that exploit delivery, shellcode execution, credential cracking or service degradation still succeeds on the victim. Payload padding preserves discrete payload exploits by appending non-executable, protocol-compliant bytes (SSH ignore messages, RFC 4253 [39]). Temporal dilation still delivers every packet, although it lowers the burst intensity of volumetric floods (Section 8); we do not test that the attack's effect on the victim is preserved.

The NIDS is an ingress biflow sensor (for example, an IPFIX collector at a border router or DMZ firewall). The adversary controls only the client endpoint, so it changes the forward features $x_2, x_4, x_6, x_7, x_{11}$ and the derived rates, while the server-generated x_3 and x_5 are immutable. Forward chaff is sent as pure ACKs with seq = SND.NXT and ack = RCV.NXT. A TCP endpoint never acknowledges a pure ACK (RFC 9293 [37], § 3.8.1), so these packets elicit no backward packets or bytes and $\Delta x_3 = \Delta x_5 = 0$.

Chaff is spaced or interleaved with forward payload and window updates so that three duplicate ACKs do not trigger fast retransmit (RFC 5681 [40], § 3.2). Window morphing edits the window field of the initial SYN consistently with window scaling (RFC 7323 [41]). For encrypted protocols such as SSH, padding is inserted at the application layer before cryptographic encapsulation (RFC 4253 [39], § 11.2).

The adversary minimizes the total operational cost

$$\min \quad \mathcal{C}(x^*, x_{\text{orig}}) = \sum_{k=1}^{4} c_k s_k \tag{7}$$

where s_k is the amount of generator k applied and c_k is its unit cost in transmission latency, bandwidth expansion and socket buffer allocation.

4. The Flow-Field Evasion (FFE) Framework

Figure 1 summarizes the framework.

4.1. Geometry of the Flow Manifold and Its Generators

The physical flow set $\mathcal{M}_{\text{flow}} = \Phi(\mathcal{P}_{\text{valid}})$ is a discrete lattice in $\mathbb{R}^{12}$. Its differential geometry is analyzed through the smooth relaxation $\widetilde{\mathcal{M}}_{\text{flow}}$ (Section 3), defined as the zero set of the conservation residual Ψ below, subject to $x_2 \geq 1$, $0 < x_6 \leq 1.1 \times 10^8$, $x_{11} \geq x_2(20 \cdot \mathbb{I}_{x_1=6} + 8 \cdot \mathbb{I}_{x_1=17})$ and $x_{11} + 20x_2 \leq x_4 \leq 1500x_2$.

$$\Psi(x) = \begin{bmatrix} x_8 - \dfrac{10^6(x_4 + x_5)}{x_6} \\ x_9 - \dfrac{10^6(x_2 + x_3)}{x_6} \\ x_{10} - \dfrac{x_4}{x_2} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} \tag{8}$$

By the Regular Value Theorem, at any point where the Jacobian $J_\Psi(x) \in \mathbb{R}^{3\times 12}$ has full rank 3, the tangent space is $T_x\widetilde{\mathcal{M}}_{\text{flow}} = \ker J_\Psi(x)$. A physically realizable velocity $\vec{v} = [\dot{x}_0, \ldots, \dot{x}_{11}]^T$ therefore satisfies the linearized conservation laws:

$$\dot{x}_8 = \frac{10^6(\dot{x}_4 + \dot{x}_5)}{x_6} - \frac{x_8}{x_6}\dot{x}_6 \tag{9}$$

$$\dot{x}_9 = \frac{10^6(\dot{x}_2 + \dot{x}_3)}{x_6} - \frac{x_9}{x_6}\dot{x}_6 \tag{10}$$

$$\dot{x}_{10} = \frac{\dot{x}_4 - x_{10}\dot{x}_2}{x_2} \tag{11}$$

An adversary controls four elementary degrees of freedom, each generating a 1-parameter flow whose push-forward is a tangent vector field. Here $\vec{e}_k$ is the kth standard basis vector of $\mathbb{R}^{12}$ (indexed from 0):

$$\begin{aligned}
\vec{v}_{\text{time}} &= \vec{e}_6 - \frac{x_8}{x_6}\vec{e}_8 - \frac{x_9}{x_6}\vec{e}_9, \\
\vec{v}_{\text{pad}} &= \vec{e}_4 + \frac{10^6}{x_6}\vec{e}_8 + \frac{1}{x_2}\vec{e}_{10}, \\
\vec{v}_{\text{chaff}} &= \vec{e}_2 + L_0\vec{e}_4 + \frac{L_0 \cdot 10^6}{x_6}\vec{e}_8 + \frac{10^6}{x_6}\vec{e}_9 + \frac{L_0 - x_{10}}{x_2}\vec{e}_{10} + H_0\vec{e}_{11}, \\
\vec{v}_{\text{win}} &= \mathbb{I}_{x_1=6}\ \text{sat}\left(\frac{W_{\text{target}} - x_7}{\epsilon_{\text{win}}}\right)\vec{e}_7,
\end{aligned} \tag{12}$$

where temporal dilation adds delay without bytes or packets, payload padding adds forward dummy bytes, chaff injection adds forward zero-payload probes, and window morphing moves the client window toward W_{target}. Here $\text{sat}(z) = \max(-1, \min(1, z))$ and $\epsilon_{\text{win}} > 0$. The probe sizes are $L_0 = 40$, $H_0 = 20$ for empty TCP segments per RFC 9293 §3.8.4 [37], $L_0 = 52$, $H_0 = 32$ with TCP timestamps [41], and $L_0 = 28$, $H_0 = 8$ for UDP [38]. Probes are classified at the ingress perimeter before server ACKs alter backward counts; bidirectional sessions are discussed in Section 8. The window field is globally Lipschitz

rather than C^∞ and depends only on x_1 and x_7. The implementation (Supplementary Algorithm S1) uses its $\epsilon_{\text{win}} \to 0$ limit, $\mathbb{I}_{x_1=6}\,\text{sign}(W_{\text{target}} - x_7)\,\vec{e}_7$, which moves by $|W_{\text{target}} - x_7|$ and lands exactly on W_{target} without overshoot.

4.2. Commutativity, Invariants and the Lie Cone

The Lie bracket of vector fields is $[X, Y] = J_Y X - J_X Y$, where J_X and J_Y are their Jacobians.

Proposition 1 (Commutativity and Integrability of Physical Flow Generators)**.** *Because protocol $x_1 \in \{6, 17\}$ is an immutable attribute ($\dot{x}_1 \equiv 0$), the ambient manifold $\widetilde{\mathcal{M}}_{flow}$ decomposes into two disjoint smooth invariant sub-manifolds: $\widetilde{\mathcal{M}}^{(6)}_{flow}$ (TCP flows) and $\widetilde{\mathcal{M}}^{(17)}_{flow}$ (UDP flows). On each component, the indicator $\mathbb{I}_{x_1=6}$ is constant, and L_0, H_0 are constant along any trajectory (the implementation selects $H_0 = 32$ iff the mean forward header $x_{11}/x_2 \geq 26$; injecting chaff of header size H_0 moves that mean toward H_0 and never across the threshold, so the selection is invariant under the flow), and denominators satisfy $x_2 \geq 1, x_6 > 0$, ensuring $\vec{v}_{time}, \vec{v}_{pad}, \vec{v}_{chaff}$ are C^∞ vector fields and $\vec{v}_{win}$ is globally Lipschitz (its brackets exist almost everywhere and vanish there, because x_7 is neither read nor written by the other three generators). The Lie brackets between all four infinitesimal packet transformation generators vanish identically on $\widetilde{\mathcal{M}}_{flow}$:*

$$[\vec{v}_i, \vec{v}_j] \equiv \mathbf{0} \in \mathbb{R}^{12}, \quad \forall i, j \in \{1, 2, 3, 4\} \tag{13}$$

Consequently, the generators span an Abelian Lie algebra $\mathfrak{g}_{flow} \cong \mathbb{R}^4$ (or $\mathbb{R}^3$ for UDP).

The proof of Proposition 1, and the Frobenius integrability corollary that follows from it (Supplementary Corollary S1), are given in the Supplementary Material (Section S1).

The admissible deformation cone is the positive span of the generators:

$$\mathcal{K}_{\text{net}}(x) = \left\{ \sum_{k=1}^{4} u_k \vec{v}_k(x) \;\middle|\; u_k \geq 0 \right\} \subset T_x \widetilde{\mathcal{M}}_{\text{flow}} \tag{14}$$

Theorem 1 (Universal Invariant Preservation)**.** *Let $x_0 \in \widetilde{\mathcal{M}}_{flow}$ be an initial network flow vector. For any measurable and locally bounded control signal $u(t) \in L^\infty_{loc}(\mathbb{R}_{\geq 0}, \mathbb{R}^4_{\geq 0})$ satisfying the admissible boundary rules ($u_2(t) \leq (1500 - L_0)u_3(t)$ whenever $x_4(t) = 1500x_2(t)$, and $u_1(t) = 0$ whenever $x_6(t) = 1.1 \times 10^8$), the trajectory $x(t)$ defined by the dynamical system:*

$$\dot{x}(t) = \sum_{k=1}^{4} u_k(t) \vec{v}_k(x(t)), \quad x(0) = x_0 \tag{15}$$

satisfies $x(t) \in \widetilde{\mathcal{M}}_{flow}$ for all $t \geq 0$, with solutions existing uniquely and globally on $[0, \infty)$. Under discrete stepping, the continuous Lie cone guides directional descent while the closed-form analytical projection operator $\mathcal{P}_{\mathcal{M}}$ (Section 4.4) eliminates discrete $\mathcal{O}(\Delta^2)$ truncation drift and enforces packet and byte integrality ($x_2 \in \mathbb{N}_{\geq 1}, x_4, x_6 \in \mathbb{N}$) and MTU clamping $x_4 \leq 1500x_2$, ensuring that the discrete state sequence remains strictly in the physical packet manifold $\mathcal{M}_{flow}$.

Proof sketch. Differentiating Ψ along the flow gives a linear homogeneous system $\dot{\Psi} = A(t)\Psi$ with diagonal, locally integrable $A(t)$, so $\Psi \equiv \mathbf{0}$ by Carathéodory's theorem [42]. The bounds $g_i \geq 0$ are affine, so the Nagumo–Brezis condition [43, 44] holds on every face and the manifold is forward

invariant. The same theorem gives existence and uniqueness of global solutions. The full proof is in the Supplementary Material (Section S1).

4.3. Operational Cost and Optimal Control

The unit costs are $c_1 = 10^{-4}$ per second of induced delay, $c_2 = 10^{-2}$ per byte of padding, $c_3 = 1.0$ per probe packet, and $c_4 = 10^{-3}$ per normalized window displacement $s_4 = |W_{\text{target}} - x_7|/65535 \in [0, 1]$. The evasion problem is an optimal control problem:

$$\min_{u(t)\geq 0} \quad \int_0^{T^*} \left(\sum_{k=1}^{4} c_k u_k(t) \right) dt \tag{16}$$

$$\text{subject to} \quad \dot{x}(t) = \sum_{k=1}^{4} u_k(t) \vec{v}_k(x(t)), \quad x(0) = x_{\text{orig}} \tag{17}$$

$$F(x(T^*)) < 0 \quad \text{(Classifier Misclassification)} \tag{18}$$

Because the generators commute (Proposition 1), the displacement depends only on $s_k = \int_0^{T^*} u_k(t)\, dt \geq 0$, and the cost $\mathcal{C}(s) = \sum_{k=1}^{4} c_k s_k$ is linear and path-independent. The tree ensemble boundary $F(x) = 0$ is piecewise constant and non-convex, so FFE greedily follows the generator with the largest directional score decrease per unit cost, $-\nabla_k / c_k$, and finishes with boundary snapping (TSBS).

4.4. Algorithmic Realization of Flow-Field Evasion

Algorithm 1 gives the master loop of Flow-Field Evasion (FFE). Starting from a flagged flow $x_0 \in \mathcal{M}_{\text{flow}}$ with $F(x_0) \geq 0$, each iteration probes the four generators, takes the admissible step with the largest score reduction per unit cost, and falls back to boundary snapping or temporal dilation when no generator lowers the score.

Algorithm 1 Flow-Field Evasion (FFE) Master Optimization Loop

Require: Classifier F, initial flow $x_0 \in \mathcal{M}_{\text{flow}}$, schema $\mathcal{S}$, step budget M, costs $\vec{c} = [c_1, \ldots, c_4]^T$, threshold $\tau = 0$
Ensure: Evasive flow $x^* \in \mathcal{M}_{\text{flow}}$ with $F(x^*) < \tau$, or FAILURE

$x \leftarrow \mathcal{P}_{\mathcal{M}}(x_0)$; $f \leftarrow F(x)$; $q \leftarrow 1$
for step $= 1$ **to** M **do**
  if $f < \tau$ **then**
    return $(x, \text{SUCCESS})$
  end if
  $\mathcal{D}, q_{\text{used}} \leftarrow \text{EstimateLieDerivatives}(F, x, \mathcal{S})$; $q \leftarrow q + q_{\text{used}}$ {Supplementary Algorithm S1}
  $k^* \leftarrow$ first maximizer of $(-\nabla_k)/c_k$ over $(k, \nabla_k, \vec{v}_k, \Delta s_k, f_k) \in \mathcal{D}$ with $\nabla_k < 0$; null if none
  if $k^* \neq$ null **then**
    $x \leftarrow \mathcal{P}_{\mathcal{M}}(x + \Delta s_{k^*}\, \vec{v}_{k^*})$; $f \leftarrow f_{k^*}$ {Cached probe, no extra query}
  else
    $x_{\text{snap}} \leftarrow \text{TSBS}(F, x, \mathcal{S})$
    if $F(x_{\text{snap}}) < f$ **then**
      $x \leftarrow x_{\text{snap}}$; $f \leftarrow F(x_{\text{snap}})$
    else
      $\Delta t \leftarrow \min(1.1 \times 10^8, \max(5 \times 10^4, 1.5\, x_6))$ {Fallback dilation; projection clamps duration at the 1.1×10^8 μs timeout}
      $x \leftarrow \mathcal{P}_{\mathcal{M}}(x + \Delta t\, \vec{v}_{\text{time}}(x))$; $f \leftarrow F(x)$; $q \leftarrow q + 1$
    end if
  end if
end for
if $f < \tau$ **then**
  return $(x, \text{SUCCESS})$
else
  return $(x, \text{FAILURE})$
end if

Supplementary Algorithm S1 estimates the directional derivatives by probing the ensemble once per generator, since ∇F vanishes almost everywhere. The probe steps are $\Delta s_1 = \max(1000,\, 0.5x_6)$ μs, $\Delta s_2 = \max(50,\, 0.25x_4)$ bytes, $\Delta s_3 = \max(1,\, \text{round}(0.5x_2))$ packets and $\Delta s_4 = |64240 - x_7|$, and the derivatives are taken per second of delay, per byte, per packet and per window unit of $\Delta s_4/65{,}535$, respectively. Every displacement is followed by the closed-form projection $\mathcal{P}_{\mathcal{M}}$ of Supplementary Algorithm S2, which re-

stores immutable features, enforces monotonicity and integer counts, bounds header and packet lengths, and recomputes the coupled rates.

4.4.1. Tree-Split Boundary Snapping (TSBS)

Algorithm 2 implements Tree-Split Boundary Snapping (TSBS), whose candidates are generated by Supplementary Algorithm S3. Because the score is piecewise constant in each split feature, a small displacement cannot cross a split threshold. In Tier 1 white-box settings, TSBS therefore extracts the active split thresholds of the ensemble, keeps per feature the threshold closest to the benign side of the current value, and retains the projected candidate that most decreases F.

GBDTs also split on coupled telemetry features, so TSBS inverts their constraints in closed form. To cross an active byte-rate threshold θ_8 or packet-rate threshold θ_9, it lengthens the flow duration to

$$x_6^* = \frac{10^6(x_4 + x_5)}{\theta_8} + \epsilon_6 \quad \text{or} \quad x_6^* = \frac{10^6(x_2 + x_3)}{\theta_9} + \epsilon_6, \tag{19}$$

with $\epsilon_6 = 10\,\mu$s; the candidate is discarded if x_6^* exceeds the $1.1 \times 10^8\,\mu$s cap. To cross a forward mean packet length threshold θ_{10} from above, TSBS inverts $x_{10} = (x_4 + NL_0)/(x_2 + N)$ and adds the smallest integer chaff count that satisfies the inequality,

$$N_{\text{chaff}} = \left\lfloor \frac{x_4 - \theta_{10} x_2}{\theta_{10} - L_0} \right\rfloor + 1 \qquad (L_0 < \theta_{10} < x_{10}). \tag{20}$$

For TCP flows, TSBS also snaps the window advertisement toward the benign OS baseline $x_7^* = 64{,}240$ and toward adjacent split partitions.

Algorithm 2 Tree-Split Boundary Snapping (TSBS)

Require: Classifier F, flow candidate $x \in \mathcal{M}_{\text{flow}}$ with $F(x) \geq \tau$ (still flagged malicious), feature schema $\mathcal{S}$, tolerance ϵ_{rel}
Ensure: Snapped flow $x^* \in \mathcal{M}_{\text{flow}}$
$x^* \leftarrow \text{copy}(x)$
active_splits $\leftarrow$ ExtractActiveSplits(F, x^*)
$\mathcal{C} \leftarrow$ GenerateCandidates$(x^*, \text{active_splits}, \epsilon_{\text{rel}})$ {Boundary-crossing pool (Supplementary Algorithm S3)}
{Evaluate projected candidates and retain best partition crossing}
$x_{\text{best}} \leftarrow x^*$; $s_{\text{best}} \leftarrow F(x^*)$
for each $x_c \in \mathcal{C}$ **do**
  $x_{\text{proj}} \leftarrow$ ProjectManifoldInvariants$(x_c, \mathcal{S})$
  if $F(x_{\text{proj}}) < s_{\text{best}}$ **then**
    $s_{\text{best}} \leftarrow F(x_{\text{proj}})$; $x_{\text{best}} \leftarrow x_{\text{proj}}$
    if $s_{\text{best}} < 0$ **then**
      break {Early exit once benign classification is achieved}
    end if
  end if
end for
return x_{best}

4.4.2. Query Accounting

Executing all $M_{\max} = 20$ iterations without early termination costs at most $Q_{\max} \leq 4M_{\max} + 1 = 81$ model queries for the baseline query and Lie-derivative probes, or ≤ 101 if stagnation fallbacks fire, plus the TSBS candidate evaluations, which are not covered by this bound; UDP flows and TCP flows whose window already equals the benign value skip window morphing and cost $q = 3$ probes per iteration. In practice the loop terminates far below this budget, and on the $N = 100$ master cohort it consumes a mean of 12.9 queries per flow, of which 10.56 are external queries and 2.38 are TSBS candidate evaluations. The 12.9 figure is deliberately conservative: a Tier 1 adversary could run the TSBS evaluations offline on its local copy of the ensemble, but the implementation counts every one of them as a query, as a strictly black-box adversary would have to. Because $K \leq 4$ and $M \leq 20$

are constants, each iteration costs $\mathcal{O}(T \cdot D)$ tree evaluations, and Table 1 compares this asymptotic cost with exact MILP and heuristic search.

5. Experimental setup

Evaluations on a single testbed have limited external validity [7, 23, 24, 25], and detectors trained on one environment transfer poorly to another [45, 46, 47]. We therefore evaluate on three corpora calibrated to the published marginal distributions of CSE-CIC-IDS2018 (enterprise) [1], UNSW-NB15 (contemporary hybrid) [14], and CIDDS-001 (cloud OpenStack) [15]. Each corpus is synthetic. It is generated to match its testbed's statistics and is not a replay of the original packet captures.

Each corpus contains 12,000 flows, of which 9,000 are benign and 3,000 malicious. The malicious flows form five families of 1,000, 500, 700, 500 and 300 flows: a SYN flood, a UDP flood, brute-force authentication against SSH and FTP, SQL injection and cross-site scripting, and low-and-slow command-and-control infiltration. The data are split into 8,000 training, 1,000 validation and 3,000 test flows. The threat model is an ingress perimeter that inspects forward telemetry before bidirectional session termination (Section 3).

The targets are XGBoost (CUDA histogram method) and LightGBM, each a 60-tree gradient-boosted ensemble of depth 6 with learning rate 0.1. Experiments ran on an NVIDIA RTX 5000 Ada GPU with 16 GB of memory, using Python 3.13.11, XGBoost 3.4.1, LightGBM 4.7.0, CatBoost 1.2.10 and scikit-learn 1.8.0.

We compare five evasion methods against FFE:

- Naive coordinate descent in $\mathbb{R}^d$ with a fixed step $\Delta = 0.05\,\sigma_f$ and no

immutability, monotonicity or rate-coupling checks.

- ZOO-NIDS [9]: zeroth-order optimization with randomly sampled coordinates, symmetric difference quotients, Adam updates and box bounds.
- HopSkipJump-NIDS [10]: a decision-based boundary attack initialized from a benign flow.
- CAA/MOEVA [12, 13]: R-NSGA-III evolutionary search with population 50 and 25 generations.
- Traffic Manipulator PSO [11]: particle swarm optimization over packet intervals and forward dummy bytes, with swarm size 50 and 37 iterations (up to 1,850 queries).
- FFE: the Lie-steering procedure (Algorithms 1–2) with step budget $M = 20$.

ASR is the fraction of attacks with $P(\text{Malicious} \mid x^*) < 0.5$. PFR is the fraction of candidate flows that pass the protocol sanity checks, the non-negativity checks and the exact coupled rate equations, that is, $x^* \in \mathcal{M}_{\text{flow}}$. For unconstrained baselines that never produce an evasive candidate, PFR is computed over evasive candidates only. Real ASR counts an attack only when it both evades and is physically feasible:

$$\text{Real ASR} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}\left[F(x_i^*) < 0 \ \wedge \ x_i^* \in \mathcal{M}_{\text{flow}}\right], \tag{21}$$

where F is the detector score and negative values denote evasion. Average queries counts the model evaluations per sample, and total time is the

wall-clock time for the cohort. The perturbation cost is the mean feature-normalized L_1 distance over the $d = 12$ features:

$$\bar{D}_{L_1}(x^*, x) = \frac{1}{d} \sum_{j=1}^{d} \frac{|x_j^* - x_j|}{\sigma_j}, \tag{22}$$

where σ_j is the empirical standard deviation of feature j in the training corpus.

Each method is run on the same cohort of 100 correctly classified malicious flows sampled at random from the test partition. The per-family and cross-dataset cohorts also contain 100 flows each. The ablation and defense studies use balanced cohorts of 20 flows per attack family, so their figures differ from the master cohort (for example, 11.6 against 12.9 queries at $M = 20$). Table 5 also trains CatBoost and Random Forest models on the same corpus. Each study trains its own model instances, so clean accuracy for the same architecture differs slightly across tables (99.70% in Table 3, 99.72% in Table 6). All experiments use a single random seed, so we report no variance.

6. Results

6.1. Master comparative benchmark

Table 2 compares FFE with the five baselines. Several baselines evade the detector but not with physically valid flows. The Real ASR of ZOO-NIDS, HopSkipJump and CAA/MOEVA falls from 12.0% to 3.0%, from 45.0% to 8.0% and from 78.0% to 48.0%, because their unconstrained mutations decouple byte and packet rates. Naive coordinate descent never evades the detector (0.0% ASR).

FFE reaches 100.0% ASR, PFR and Real ASR. At $N = 100$, the 95% Wilson score interval for Real ASR is [96.3%, 100.0%]. Theorem 1 guarantees that continuous trajectories in the Lie cone remain on the relaxed manifold; on the discrete schema the projection after each step enforces the same relations, so PFR measures conformance to the 12-feature schema and is not an independent test of live packets.

FFE needs 12.9 queries per flow on average, against 680.0 for HopSkipJump, 1,240.0 for CAA/MOEVA and 1,850.0 for PSO, which is 53×, 96× and 143× fewer queries. FFE evaluates directional Lie derivatives along its four generators rather than searching the 12-dimensional feature space blindly.

FFE processes the 100 flows in 4.37 s, or 43.7 ms per flow, against 420.15 s for PSO, a 96× speedup in algorithmic runtime. These wall-clock times are dominated by single-row CUDA inference. Repeated runs of the same benchmark gave totals between 1.5 s and 4.4 s, so only order-of-magnitude conclusions should be drawn. The 43.7 ms excludes the network delay that FFE adds to the packet stream.

FFE also has the smallest perturbation, with mean $\bar{D}_{L_1}$ of 0.251, against 0.342 to 0.512 for ZOO-NIDS, HopSkipJump, CAA/MOEVA and PSO. Naive coordinate descent reaches 204.605.

6.2. Per-family and cross-dataset validation

Table 3 reports FFE by attack family on CSE-CIC-IDS2018 (Panel A) and by dataset and target model (Panel B).

FFE succeeds on every family in Panel A. The dominant generator follows each family's signature. Temporal dilation dominates the two volumetric

floods, whose byte and packet rates the trees split on. Padding dominates brute-force authentication, whose fixed packet lengths sit near a split boundary. Chaff injection dominates web exploits, and window morphing dominates infiltration, which moves the window to the standard client value of 64,240, the benign value in our synthetic generator. The unweighted family mean is 11.6 queries and 59.7 ms. The master cohort needs 12.9 queries because it is drawn from the mixed test partition and has a different family mix.

Across the three corpora and both target models, FFE evades every flow with 100.0% ASR, PFR and Real ASR. Clean accuracy ranges from 99.63% to 99.97%, so the targets are strong detectors. Queries range from 6.8 (CIDDS-001, XGBoost) to 17.8 (UNSW-NB15, LightGBM), and latency from 8.4 ms to 66.0 ms. UNSW-NB15 is the costliest corpus and CIDDS-001 the cheapest. CUDA XGBoost is the slowest model on all three corpora because each single-row query pays a host-to-device transfer, and its latencies vary between runs.

6.3. Ablations and transferability

Table 4 reports three ablations on the CUDA XGBoost target of Section 5, using the balanced cohort of $N = 100$ detected malicious flows, where one flow corresponds to one percentage point. Panel A varies the step budget M, Panel B disables generators, and Panel C varies the cost regime.

ASR rises from 41.0% at $M = 1$ to 95.0% at $M = 5$ and reaches 100.0% at $M = 10$, with 11.6 queries and a normalized cost of $\bar{D}_{L_1} = 0.218$. Budgets above 10 change neither ASR, queries nor cost, so $M = 20$ and $M = 50$ are omitted from the table. The mean cost grows from 0.137 at $M = 1$ to 0.218

and then stays flat.

A disabled generator is removed from the directional probes of Supplementary Algorithm S1 and from the TSBS candidate set of Algorithm 2. Temporal dilation alone is the strongest single generator at 67.0%, but it adds 32.5 s of flow duration on average. The best pair, time with chaff, reaches 82.0% at 16.19 s, and no proper subset reaches 100.0%. The full cone reaches 100.0% at 11.6 queries and 0.40 s of added delay. PFR remains 100.0% for every subset because the projection enforces the schema relations whichever generators are active.

Algorithm 1 selects generators by the score $(-\nabla_k)/c_k$, so the cost vector $\vec{c}$ sets the trade-off. Regime A uses the default cost vector of Section 4.3. Penalizing delay in Regime B cuts the mean added delay from 396 ms to 133 ms at a similar query cost (11.3 vs. 11.6), while the normalized cost rises from 0.218 to 0.246. Regime C is nearly identical to Regime A, with 396 ms in both. All four regimes reach 100.0% ASR, and $\vec{v}_{\text{time}}$ is the dominant generator in each, where dominance counts only the directional step of Algorithm 1 and TSBS edits are not attributed to a generator. Varying the relative snapping offset ϵ_{rel} from 0 to 1.0 leaves ASR and PFR at 100.0% and lowers mean queries only from 11.6 to 11.4.

In many operational settings the attacker cannot query the deployed model directly. Table 5 therefore reports the attack success rate of flows crafted on a source model and evaluated on each target. Off-diagonal entries range from 69% to 94%, so flows crafted on one tree ensemble usually also evade the others. We attribute this to the shared reliance of tree ensembles on the same volumetric features that the generators modify, but we did not

test this mechanism directly.

7. Defenses

Standard boosting chooses split features by training-loss reduction alone, so features that separate the classes widely in the synthetic corpora, such as duration and the rates, sit near the root although an attacker can shift them by delaying or padding packets. We evaluate malleability-penalized splitting (MPS), which discourages splits on malleable features, and Lie-guided adversarial training (LGAT), which trains on flows generated along the Lie generators.

7.1. Malleability-penalized tree splitting

Following the robust split formulations of Chen et al. [32] and Calzavara et al. [34], the split score for feature j is regularized by its physical malleability index.

Definition 1 (Feature Physical Malleability Index)**.** The physical malleability index $M_{\text{mal}}(j)$ of feature j is defined as the empirical expectation over the training distribution of the maximum dimensionless variation induced by the Lie Cone $\mathcal{K}_{\text{net}}$:

$$M_{\text{mal}}(j) = \mathbb{E}_{x \sim \mathcal{D}_{\text{train}}} \left[\max_{k \in \{1,\dots,4\}} \frac{\delta s_k |\vec{v}_k(x)_j|}{\sigma_j} \right] \tag{23}$$

where σ_j is the empirical standard deviation of feature j across the training corpus $\mathcal{D}_{\text{train}}$, and δs_k denotes the characteristic single-step perturbation scale for generator k ($\delta s_1 = 10^4\,\mu\text{s}, \delta s_2 = 100\,\text{B}, \delta s_3 = 1\,\text{pkt}, \delta s_4 = 10^3\,\text{B}$).

The robust split selection objective is

$$\mathrm{Gain}_{\mathrm{robust}}(j,\theta) = \Delta\mathcal{L}(j,\theta) - \alpha_{\mathrm{mal}} \cdot \overline{\Delta\mathcal{L}} \cdot M_{\mathrm{mal}}(j) \tag{24}$$

where $\Delta\mathcal{L}(j,\theta)$ is the impurity gain of split (j,θ), $\overline{\Delta\mathcal{L}}$ is the baseline mean impurity gain across candidate splits at the node, ensuring dimensional consistency, and $\alpha_{\mathrm{mal}} \geq 0$ is a dimensionless hyperparameter governing the trade-off between clean training loss and malleability robustness.

The experiments do not optimize this objective. The MPS model instead masks duration (x_6) and the two rate features (x_8, x_9) to zero in the training, test and attack inputs, which is the $\alpha_{\mathrm{mal}} \to \infty$ limit for those features. The mask is fixed by hand rather than computed from M_{mal}, and other features are unpenalized. Because the masked columns are constant, trees cannot split on them, so the attacker affects MPS models only through unmasked features.

7.2. Lie-guided adversarial training

Standard adversarial training assumes an L_p ball around each input [48], which in network security generates physically invalid samples that degrade clean accuracy [35, 7]. LGAT instead restricts the inner maximization to the physical flow manifold, solving

$$\min_{\Theta} \frac{1}{N} \sum_{i=1}^{N} \max_{u \in \mathcal{U}_{\mathrm{budget}}} \mathcal{L}\left(F_{\Theta}\left(\mathcal{A}_{\mathrm{cone}}\left(x_i, u\right)\right), y_i\right) \tag{25}$$

where $\mathcal{A}_{\mathrm{cone}}(x,u)$ is the flow reached from x along the integral curves of the Lie generators under the control u, and $\mathcal{U}_{\mathrm{budget}}$ is the set of nonnegative controls whose cost $\sum_{k=1}^{4} c_k \int_0^{T_{\max}} u_k(\tau)\, d\tau$ does not exceed Γ.

Supplementary Algorithm S4 gives the general scheme, in which malicious flows are replaced by FFE outputs at step budget $M_{\text{train}} = 5$ in every boosting round, with $\beta = 0.5$. The experiments use a single-round instance: adversarial examples are generated once, against the undefended model, for a random 50% of the malicious training flows and appended to the training set, and the ensemble uses standard split scoring.

7.3. Evaluation

We compared an undefended XGBoost model with three hardened variants on the CSE-CIC-IDS2018 calibrated corpus, each with 60 trees of depth 6 and learning rate 0.1. All models are attacked on the same cohort of 100 flows that the undefended model flags, using FFE at step budget $M = 5$ (Table 6).

Combining MPS with LGAT reduces FFE success from 95.0% to 56.0% and raises robust recall from 5.0% to 44.0%. The clean accuracy cost is 2.42 percentage points (99.72% to 97.30%), with clean ROC-AUC of 99.69%. The attack budget explains the undefended figure: at $M = 5$ FFE succeeds on 95.0% of the cohort, whereas Section 6 reports 100.0% at the larger budget. The residual 56.0% success shows that the defense is partial.

8. Discussion

Feature-space attacks are fast, but their output is not guaranteed to be a physically realizable flow (Section 2). FFE pays for feasibility with a different structure. Its generators act on the flow manifold, so trajectories in the continuous Lie cone satisfy the conservation and rate-coupling invariants exactly, and a projection step restores them on the discrete flow schema. Feasibility

is therefore 100% in every experiment, but this figure measures conformance to the 12-feature schema, not an independent test on live packets.

Against population and swarm methods, the advantage is query efficiency. CAA/MOEVA and Traffic Manipulator PSO need 1,240 and 1,850 queries, against 12.9 for FFE on the primary benchmark. The saving comes from the ensemble's own geometry: TSBS snaps flows onto split boundaries instead of searching blindly. Compared with MILP and SAT/SMT encodings, FFE does not encode the whole ensemble as a constraint problem, whose worst-case cost is $2^{O(DT)}$ for MILP with T trees of depth D. It also accepts nonlinear invariants such as $x_8 = 10^6(x_4+x_5)/x_6$, which a purely linear encoding cannot express.

The method has clear limits. TSBS requires Tier-1 white-box access to the split thresholds. Under Tier-2 access the attack depends on probes and transfer, and transferability is high but not complete, with off-diagonal rates of 69–94%. Traffic Manipulator operates on captured PCAP files, whereas our packet realization is a Scapy/NFQueue proxy that has not been replayed at line rate. Control is restricted to forward traffic, because backward server features are treated as immutable. Deep packet inspection is out of scope, since payload padding alone does not defeat signature matches. Rate dilution also lowers the intensity of volumetric floods, so an attacker trades burst rate for stealth. The corpora are synthetic and calibrated to published marginal distributions rather than replayed captures, and the strongest defense we tested still leaves 56.0% of FFE attacks successful.

Each generator has a protocol-compliant packet-level realization: sender pacing for temporal dilation, padding in protocol fields or ignored application

messages such as `SSH_MSG_IGNORE` sent before encryption [39], payload-free pure ACK or keep-alive chaff [37, 41] or zero-length UDP datagrams [38], and window adjustment during the handshake. A proxy then updates sequence numbers and recomputes the IPv4 and transport checksums [36, 37, 38].

We ran all experiments on synthetic benchmarks in a controlled setting. The aim is to expose a weakness of GBDT detectors before it is exploited, and the results support the robust training of Section 7.

9. Conclusions

This article has shown that tree-ensemble NIDS with near-perfect benchmark accuracy are vulnerable to evasion that respects flow physics. Flow-Field Evasion models four packet-level actions as infinitesimal generators on the flow manifold. Trajectories in the continuous Lie cone preserve the conservation invariants, and a projection enforces them on the discrete schema. FFE reaches 100% real attack success against XGBoost at 12.9 queries per flow, and 100% on three corpora and two gradient-boosting libraries at 6.8–17.8 queries per flow, whereas baseline real attack success ranges from 0.0% to 82.0% in our comparison. Feature masking combined with single-round FFE adversarial training reduces FFE success from 95.0% to 56.0% at a five-step budget against a non-adaptive attacker, at a 2.42-point clean-accuracy cost. This is a partial defense, and adaptive attackers were not evaluated. Future work will address line-rate replay on hardware and certified robustness bounds for adversarially trained ensembles.

CRediT authorship contribution statement

Quang-Vinh Dang: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing – original draft, Writing – review & editing. [AUTHOR: confirm]

Declaration of competing interest

The author declares no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. [AUTHOR: confirm]

Funding

[AUTHOR: confirm]

Data availability

The code and the synthetic corpus generators, together with the result files for FFE, the ablations, the transferability study and the defenses, are available at https://github.com/vinhqdang/adversarial_ids and archived at [AUTHOR: Zenodo DOI]. All data are synthetic and generated by the scripts. [AUTHOR: state the provenance of the ZOO, HopSkipJump, CAA/MOEVA and PSO rows of Table 2 before submission.]

Acknowledgements

[AUTHOR]

References

[1] I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, Toward generating a new intrusion detection dataset and intrusion traffic characterization, in: Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP), 2018, pp. 108–116. doi:10.5220/0006639801080116.

[2] T. Chen, C. Guestrin, XGBoost: A scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016, pp. 785–794. doi:10.1145/2939672.2939785.

[3] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, LightGBM: A highly efficient gradient boosting decision tree, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 30, 2017, pp. 3146–3154.

[4] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, A. Gulin, CatBoost: Unbiased boosting with categorical features, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 31, 2018, pp. 6638–6648.

[5] R. Sommer, V. Paxson, Outside the closed world: On using machine learning for network intrusion detection, in: Proceedings of the 2010 IEEE Symposium on Security and Privacy (S&P), 2010, pp. 305–316. doi:10.1109/SP.2010.25.

[6] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, F. A. Wichmann, Shortcut learning in deep neural networks, Nature Machine Intelligence 2 (11) (2020) 665–673. doi:10.1038/s42256-020-00257-z.

[7] G. Apruzzese, M. Andreolini, L. Ferretti, M. Marchetti, M. Colajanni, Modeling realistic adversarial attacks against network intrusion detection systems, ACM Digital Threats: Research and Practice 3 (3) (2022) 1–19. doi:10.1145/3469659.

[8] F. Pierazzi, F. Pendlebury, J. Cortellazzi, L. Cavallaro, Intriguing properties of adversarial ML attacks in the problem space, in: Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P), 2020, pp. 1332–1349. doi:10.1109/SP40000.2020.00073.

[9] P.-Y. Chen, H. Zhang, Y. Sharma, J. Yi, C.-J. Hsieh, ZOO: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models, in: Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security (AISEC), 2017, pp. 15–26. doi:10.1145/3128572.3140448.

[10] J. Chen, M. I. Jordan, M. J. Wainwright, HopSkipJumpAttack: A query-efficient decision-based attack, in: Proceedings of the 41st IEEE Symposium on Security and Privacy (S&P), 2020, pp. 1277–1294. doi:10.1109/SP40000.2020.00045.

[11] D. Han, Z. Wang, Y. Zhong, W. Chen, J. Yang, S. Lu, X. Shi, X. Yin, Evaluating and improving adversarial robustness of machine learning-based network intrusion detectors, IEEE Journal on Selected Areas in Communications 39 (8) (2021) 2632–2647. doi:10.1109/JSAC.2021.3087242.

[12] T. Simonetto, S. Ghamizi, M. Cordy, Constrained adaptive attack: Effective adversarial attack against deep neural networks for tabular data, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 37, 2024, pp. 8910–8932. doi:10.48550/arXiv.2406.00775.

[13] S. Ghamizi, M. Cordy, M. Papadakis, Y. L. Traon, Search-based adversarial testing and improvement of constrained credit scoring systems, in: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE), 2020, pp. 1089–1100. doi:10.1145/3368089.3409739.

[14] N. Moustafa, J. Slay, UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set), in: Proceedings of the 2015 Military Communications and Information Systems Conference (MilCIS), 2015, pp. 1–6. doi:10.1109/MilCIS.2015.7348942.

[15] M. Ring, S. Wunderlich, D. Grüdl, D. Landes, A. Hotho, Flow-based benchmark data sets for intrusion detection, in: Proceedings of the 16th European Conference on Cyber Warfare and Security (ECCWS), 2017, pp. 361–369.

[16] Q.-V. Dang, Using machine learning for intrusion detection systems, Computing and Informatics 41 (1) (2022) 12–33. doi:10.31577/cai_2022_1_12.

[17] Q.-V. Dang, Multi-layer intrusion detection on the USB-IDS-1 dataset, in: Hybrid Intelligent Systems (HIS 2022), Vol. 647 of Lecture Notes in Networks and Systems, Springer, Cham, 2023, pp. 1114–1121. doi:10.1007/978-3-031-27409-1_102.

[18] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, R. Fergus, Intriguing properties of neural networks, in: Proceedings of the International Conference on Learning Representations (ICLR), 2014.

[19] I. J. Goodfellow, J. Shlens, C. Szegedy, Explaining and harnessing adversarial examples, in: International Conference on Learning Representations (ICLR), 2015. doi:10.48550/arXiv.1412.6572.

[20] N. Papernot, P. McDaniel, S. Jha, M. Fredrikson, Z. B. Celik, A. Swami, The limitations of deep learning in adversarial settings, in: Proceedings of the 2016 IEEE European Symposium on Security and Privacy (EuroS&P), 2016, pp. 372–387. doi:10.1109/EuroSP.2016.36.

[21] N. Carlini, D. Wagner, Towards evaluating the robustness of neural networks, in: Proceedings of the 38th IEEE Symposium on Security and Privacy (S&P), 2017, pp. 39–57. doi:10.1109/SP.2017.49.

[22] M. Verkerken, L. D'hooge, B. Volckaert, F. D. Turck, G. Apruzzese, "What is the problem space?" Defining host-space adversarial perturbations against network intrusion detection systems, in: Proceedings of the 21st ACM ASIA Conference on Computer and Communications Security (ASIA CCS), 2026, pp. 1–16. doi:10.48550/arXiv.2404.14876.

[23] G. Apruzzese, H. S. Anderson, L. Cavallaro, "Real attackers don't compute gradients": Bridging the gap between adversarial ML research and practice in cybersecurity, in: Proceedings of the 1st IEEE Conference on Secure and Trustworthy Machine Learning (SaTML), 2023, pp. 339–364. doi:10.1109/SaTML54575.2023.00031.

[24] K. He, D. D. Kim, M. R. Asghar, Adversarial machine learning for network intrusion detection systems: A comprehensive survey, IEEE Communications Surveys & Tutorials 25 (1) (2023) 538–566. doi:10.1109/COMST.2022.3233793.

[25] J. Vitorino, I. Praça, E. Maia, SoK: Realistic adversarial attacks and defenses for intelligent network intrusion detection, Computers & Security 134 (2023) 103433. doi:10.1016/j.cose.2023.103433.

[26] T. Simonetto, S. Ghamizi, M. Cordy, TabularBench: Benchmarking adversarial robustness for tabular deep learning in real-world use-cases, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 37, 2024, pp. 14201–14225. doi:10.48550/arXiv.2408.07579.

[27] Z. Lin, Y. Shi, Z. Xue, IDSGAN: Generative adversarial networks for attack generation against intrusion detection, arXiv preprint arXiv:1809.02077 (2018). doi:10.48550/arXiv.1809.02077.

[28] A. Kantchelian, J. D. Tygar, A. Joseph, Evasion and hardening of tree ensemble classifiers, in: Proceedings of the 33rd International Conference on Machine Learning (ICML), 2016, pp. 2387–2396.

[29] A. Ignatiev, Y. Izza, P. J. Stuckey, J. Marques-Silva, Using MaxSAT for efficient explanations of tree ensembles, in: Proceedings of the 36th AAAI Conference on Artificial Intelligence, Vol. 36, 2022, pp. 3776–3785. doi:10.1609/aaai.v36i4.20292.

[30] Y. Izza, A. Ignatiev, J. Marques-Silva, On tackling explanation redundancy in decision trees, Journal of Artificial Intelligence Research 75 (2022) 261–321. doi:10.1613/jair.1.13575.

[31] C. Zhang, H. Zhang, C.-J. Hsieh, An efficient adversarial attack for tree ensembles, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, 2020, pp. 16165–16176.

[32] H. Chen, H. Zhang, S. Si, Y. Li, D. Boning, C.-J. Hsieh, Robust decision trees against adversarial examples, in: Proceedings of the 36th International Conference on Machine Learning (ICML), 2019, pp. 1122–1131.

[33] M. Andriushchenko, M. Hein, Provably robust boosted decision stumps and trees against adversarial attacks, in: Advances in Neural Information Processing Systems (NeurIPS), Vol. 32, 2019, pp. 12997–13008.

[34] S. Calzavara, C. Lucchese, G. Tolomei, S. A. Abebe, S. Orlando, Treant: training evasion-aware decision trees, Data Mining and Knowledge Discovery 34 (5) (2020) 1390–1420. doi:10.1007/s10618-020-00694-9.

[35] M. J. Hashemi, E. Keller, Enhancing robustness against adversarial examples in network intrusion detection systems, in: Proceedings of the 2020 IEEE Conference on Network Function Virtualization and Software Defined Networks (NFV-SDN), 2020, pp. 37–43. doi:10.1109/NFV-SDN50289.2020.9289869.

[36] J. Postel, Internet protocol, RFC 791, RFC Editor (1981). doi:10.17487/RFC0791.

[37] W. Eddy, Transmission control protocol (TCP) specification, RFC 9293, RFC Editor (2022). doi:10.17487/RFC9293.

[38] J. Postel, User datagram protocol, RFC 768, RFC Editor (1980). doi:10.17487/RFC0768.

[39] T. Ylonen, C. Lonvick, The secure shell (SSH) transport layer protocol, RFC 4253, RFC Editor (2006). doi:10.17487/RFC4253.

[40] M. Allman, V. Paxson, E. Blanton, TCP congestion control, RFC 5681, RFC Editor (2009). doi:10.17487/RFC5681.

[41] D. Borman, B. Braden, V. Jacobson, R. Scheffenegger, TCP extensions for high performance, RFC 7323, RFC Editor (2014). doi:10.17487/RFC7323.

[42] C. Carathéodory, Vorlesungen über reelle Funktionen, B. G. Teubner, Leipzig, 1918.

[43] M. Nagumo, Über die Lage der Integralkurven gewöhnlicher Differentialgleichungen, Proceedings of the Physico-Mathematical Society of Japan. 3rd Series 24 (1942) 551–559. doi:10.11429/ppmsj1919.24.0_551.

[44] H. Brézis, On a characterization of flow-invariant sets, Communications on Pure and Applied Mathematics 23 (2) (1970) 261–263. doi:10.1002/cpa.3160230211.

[45] Q.-V. Dang, Detecting intrusion using multiple datasets in software-defined networks, in: Future Data and Security Engineering. Big Data, Security and Privacy, Smart City and Industry 4.0 Applications (FDSE 2022), Vol. 1688 of Communications in Computer and Information Science, Springer, Singapore, 2022, pp. 739–746. doi:10.1007/978-981-19-8069-5_55.

[46] Q.-V. Dang, Learning to transfer knowledge between datasets to enhance intrusion detection systems, in: Computational Intelligence, Vol. 968 of Lecture Notes in Electrical Engineering, Springer, Singapore, 2023, pp. 39–46. doi:10.1007/978-981-19-7346-8_4.

[47] Q.-V. Dang, D. Le, M. N. Dinh, N.-S.-A. Nguyen, T.-B.-D. Vo, ST-Fedxids: A spatiotemporal federated explainable intrusion detection system with drift-adaptive graph learning for high-density public WiFi environments, International Journal of Advances in Signal and Image Sciences 12 (1) (2026) 873–886. doi:10.29284/zcrk2p65.

[48] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, A. Vladu, Towards deep learning models resistant to adversarial attacks, in: International Conference on Learning Representations (ICLR), 2018. doi:10.48550/arXiv.1706.06083.

Table 1: Representative evasion, verification and defense paradigms for tabular and network telemetry. Physical feasibility reports the empirical PFR of this benchmark where it was evaluated (Section 6).

Method	**Target model**	**Engine**	**Queries**	**Coupled invariants**	**Physical feasibility**
Kantchelian et al. [28]	GBDT, random forest	MILP (worst case $2^{\mathcal{O}(DT)}$)	0 (white-box)	Unsupported (linear only)	Unconstrained
Zhang et al. [31] (LT-Attack)	Tree ensembles	Continuous leaf approximation	$O(N_{\text{iter}})$	Not modeled	Unconstrained
Ignatiev et al. [29]; Izza et al. [30]	GBDT, decision trees	MaxSAT / SAT verification	0 (white-box)	Not modeled	Not applicable (explanation)
Chen et al. (ZOO) [9]	Black-box classifiers	Zeroth-order coordinate descent	$O(d \cdot N_{\text{iter}})$	Not modeled	25.0% (empirical)
Chen et al. (HSJ) [10]	Black-box decision	Boundary random walk	$O(10^3)$	Not modeled	17.8% (empirical)
Lin et al. (IDS-GAN) [27]	Black-box NIDS	Generative adversarial network	$O(N_{\text{gen}})$	Learned distribution	Approximated (feature space)
Ghamizi et al. (MOEVA) [13]	Constrained tabular	Multi-objective genetic search	$\leq$1,250	Heuristic penalty	Partial
Simonetto et al. (CAA) [12]	Tabular models	CAPGD with MOEVA	1,240	Approximate penalty	Partial (61.5% empirical)
Han et al. (Traffic Manipulator) [11]	Black-box NIDS	PSO on PCAP files	$O(10^3$–$10^4)$	Packet re-evaluation	100% (packet mutation)
Flow-Field Evasion (ours)	**XGBoost, LightGBM**	**Lie vector field steering**	**$\leq 4M_{\max}+1$ probes plus TSBS (mean 12.9)**	**Exact closed-form**	**100.0% (schema-checked)**

Note: T trees, D depth, $M_{\max} = 20$, $K = 4$ generators. MOEVA uses $G = 25$, $P = 50$ ($\leq$1,250 queries) and PSO uses $I = 37$, $S = 50$ ($\leq$1,850 queries). Empirical PFR values are from Table 2.

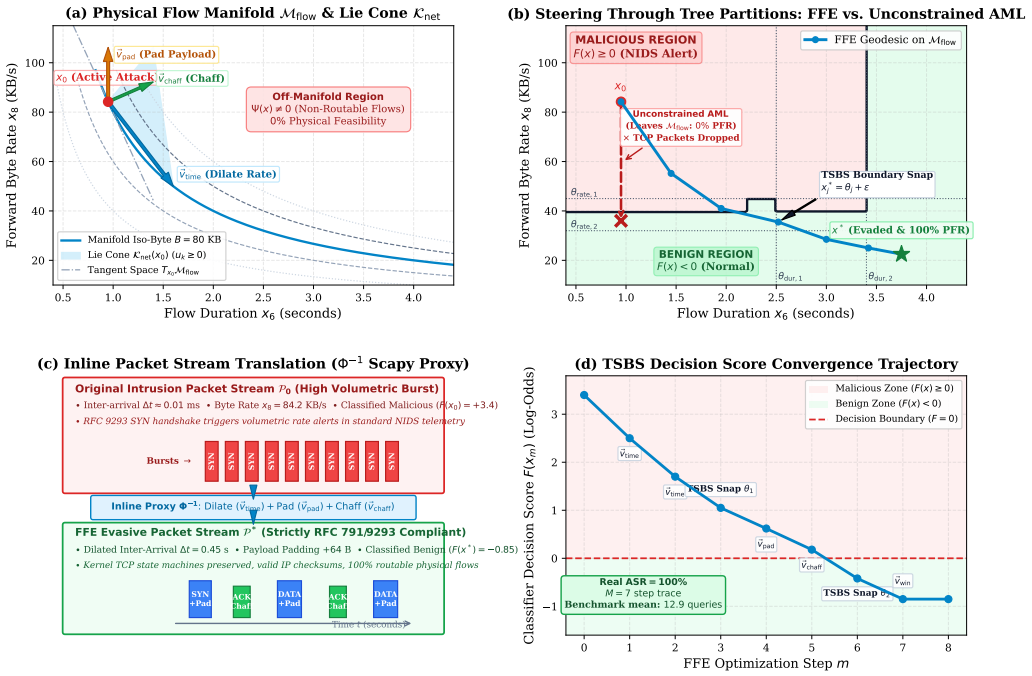


Figure 1: Architecture of the Flow-Field Evasion (FFE) framework. **(a)** The physical conservation constraints define the flow manifold $\mathcal{M}_{\text{flow}}$, and the generators span a forward Lie cone that keeps trajectories on it. **(b)** Axis-aligned tree partitions push unconstrained feature-space perturbations off the manifold, whereas FFE moves along the manifold and finishes with tree-split boundary snapping (TSBS). **(c)** Packet-level realization of the flow transformations through an inline proxy. **(d)** Decision score descent over the steps of an attack trace.

Table 2: Adversarial evasion benchmark against CUDA XGBoost on the calibrated CSE-CIC-IDS2018 corpus ($N = 100$ attacks).

Method	ASR (%)	PFR (%)	Real ASR (%)	Avg Queries	Avg Cost ($\bar{D}_{L_1}$)	Total Time (s)
Unconstrained Feature-Space	0.0	N/A†	0.0	32.8	204.605	6.00
ZOO-NIDS [9]	12.0	25.0	3.0	142.5	0.382	34.12
HopSkipJump-NIDS [10]	45.0	17.8	8.0	680.0	0.512	145.80
CAA / MOEVA [12, 13]	78.0	61.5	48.0	1,240.0	0.342	310.50
Traffic Manipulator PSO [11]	82.0	100.0	82.0	1,850.0	0.485	420.15
FFE (ours)	**100.0**	**100.0**	**100.0**	**12.9**	**0.251**	**4.37**

†PFR is evaluated over evasive candidates (0/0); unconstrained coordinate descent without manifold projection fails to produce an evasive candidate. Costs and metric definitions are given in Section 5.

Table 3: Flow-Field Evasion by attack family on CSE-CIC-IDS2018 against CUDA XGBoost (Panel A, $N = 100$ flows per family) and across datasets and target models (Panel B, $N = 100$ flows per row). ASR = PFR = Real ASR = 100.0% in every row.

Panel A: attack family (CSE-CIC-IDS2018, XGBoost)

Attack family	**Avg Queries**	**Dominant generator**	**Avg Latency (ms)**
Family 1: SYN flood	20.1	$\vec{v}_{\text{time}}$	92.2
Family 2: UDP flood	14.0	$\vec{v}_{\text{time}}$	72.2
Family 3: Brute force	5.2	$\vec{v}_{\text{pad}}$	30.5
Family 4: Web exploits	13.7	$\vec{v}_{\text{chaff}}$	72.4
Family 5: Infiltration	5.0	$\vec{v}_{\text{win}}$	31.2
Unweighted mean (5 families)	**11.6**	–	**59.7**

Panel B: dataset and target model

Dataset / model		**Clean Acc (%)**	**Clean AUC (%)**	**Avg Queries**	**Avg Latency (ms)**
CSE-CIC-IDS2018	XGBoost	99.70	99.99	12.9	57.4
CSE-CIC-IDS2018	LightGBM	99.63	99.99	15.8	13.2
UNSW-NB15	XGBoost	99.67	100.00	14.3	66.0
UNSW-NB15	LightGBM	99.77	100.00	17.8	15.8
CIDDS-001	XGBoost	99.93	100.00	6.8	24.4
CIDDS-001	LightGBM	99.97	100.00	9.3	8.4

Table 4: Ablations of Flow-Field Evasion on the CUDA XGBoost target. Budgets $M = 20$ and $M = 50$ give the same results as $M = 10$ and are omitted. PFR = 100% in every row; balanced cohort $N = 100$ (20 flows per family). In Panel C, the generator share is the fraction of steps taken along the dominant generator.

Panel A: step budget

Step budget M	**ASR (%)**	**Avg queries**	**Avg cost** $\bar{D}_{L_1}$
$M = 1$	41.0%	5.5	0.137
$M = 2$	48.0%	8.0	0.183
$M = 3$	64.0%	9.9	0.206
$M = 5$	95.0%	11.5	0.218
$M = 10$	100.0%	11.6	0.218

Panel B: generator subsets

Generator subset	**ASR (%)**	**Avg queries**	**Added duration (s)**
Time only ($\vec{v}_{\text{time}}$)	67.0%	25.9	32.53
Padding only ($\vec{v}_{\text{pad}}$)	22.0%	4.4	0.00
Chaff only ($\vec{v}_{\text{chaff}}$)	23.0%	4.7	0.00
Window only ($\vec{v}_{\text{win}}$)	40.0%	2.4	0.00
Time + padding	76.0%	32.4	24.65
Time + chaff	82.0%	31.5	16.19
Padding + chaff	43.0%	7.8	0.00
Complete Lie cone	**100.0%**	**11.6**	**0.40**

Panel C: cost regimes

Cost regime	**ASR (%)**	**Avg queries**	**Mean delay (ms)**
A (default): $\vec{v}_{\text{time}}$, 53%	100.0%	11.6	396
B (latency): $\vec{v}_{\text{time}}$, 44%	100.0%	11.3	133
C (free delay): $\vec{v}_{\text{time}}$, 52%	100.0%	11.6	396
D (uniform): $\vec{v}_{\text{time}}$, 47%	100.0%	11.5	214

Table 5: Cross-Architecture Adversarial Transferability Matrix (Attack Success Rate %)

Source \ target	**XGBoost (CUDA)**	**LightGBM**	**CatBoost**	**Random Forest**
XGBoost (CUDA)	**100.0%**	82.0%	94.0%	79.0%
LightGBM	92.0%	**99.0%**	94.0%	88.0%
CatBoost	79.0%	71.0%	**100.0%**	69.0%
Random Forest	88.0%	75.0%	92.0%	**100.0%**

Table 6: Defensive hardening against Flow-Field Evasion on CSE-CIC-IDS2018 (XGBoost; calibrated corpus; cohort $N = 100$; attack budget $M = 5$; all values in %). Robust recall is $100 - \text{ASR}$.

Model	Clean acc.	Clean AUC	Robust recall	FFE ASR
Undefended	99.72	99.99	5.0	95.0
MPS-hardened	97.80	99.81	23.0	77.0
LGAT-hardened	99.25	99.97	12.0	88.0
MPS + LGAT	97.30	99.69	44.0	56.0